# MediMesh: Intelligent Hospital Flow Optimization

RCOS

Ramji Ravi, Tim Blyumin

RPI

## Introduction

"MediMesh is a smart healthcare platform that makes advanced hospital management and predctive insights accessible and straightforward for medical staff."

- Healthcare operations involve vast amounts of complex data that's hard to handle manually
- Medical teams require quicker, smarter methods to anticipate patient needs and allocate resources effectively
- Current hospital systems are frequently rigid, costly, and missing modern predictive features
- Real-time insights are crucial for optimizing bed utilization and reducing patient wait times

## Semester Goals

- Build a predictive model to forecast ER wait times and bed occupancy using historical and synthetic hospital data
- Develop a real-time dashboard for hospital administrators to visualize patient flow, staff load, and resource utilization
- Design a modular system architecture that could scale into a product for smaller hospitals
- Train and evaluate models (Random Forest) to achieve high accuracy in time-series forecasting
- Produce comprehensive documentation and a containerized environment to allow for easy open-source contribution and deployment

Technology used:

- Frontend
  - React.js
  - Tailwind CSS
  - Lucide React
  - Recharts
  - Socket.io Client
- Backend
  - Python
    - FastAPI
    - SQLAlchemy
    - Scikit-learn
    - Pandas
    - Pydantic
  - PostgreSQL
  - Docker

## Progress and Results

- Integrated JWT clinician authentication. Protected sensitive patient data.
- Calculated department load instantly. Validated sub-second monitoring logic.
- Triggered alerts at 90% capacity. Used WebSockets for instant broadcast. Eliminated refresh lag.

---

- Developed a complete FastAPI backend for hospital entities and predictive analytics endpoints
- Integrated Random Forest models for wait time and bed occupancy forcasting
- Implemented modular architecture with SQLAlchemy, PostgreSQL database, and health monitoring features
- Containerized the application with Docker for easy deployment

## Conclusions

- Proved event-driven design works. Reduced technical barriers to observability.
- Sub-second alerts prevent crises. Enabled faster resource allocation decisions.
- Established secure foundation. Ready for ML model scaling. Supports future dashboard expansion.

---

- Gained fundamental knowledge on SQLAlchemy, PostgreSQL, FastAPI, and integrating machine learning for predictions
- Worked towards establishing a foundation for data-driven hospital management with real-time visualization

## Acknowledgements

Team contributions:

- Ramji Ravi
  - Team Lead
- Tim Blyumin
  - Team Member

Special thanks to:

- Professor Wes Turner
- RCOS coordinators
- and the rest of the mentors and faculty